공고 기반 면접 준비

> 목적: 지원 공고가 실제로 확인하려는 역량 축과, 이 프로젝트를 그 기준에 맞춰 어떻게 설명할지 정리한 문서;

1. 공고가 중요하게 보는 핵심 축

1-1. 구조 설계와 확장성

- 구현 전에 구조를 먼저 고민하는지;
- 당장 동작하는 코드보다 이후 변경 비용을 줄이는 설계를 하는지;
- 컴포넌트, 데이터 흐름, 상태 분리를 설명할 수 있는지;

답변 예시:

- `이 프로젝트는 거래 입력 하나가 통계, 예산, 자산 현황까지 연결되는 구조라서, 화면 단위보다 데이터 흐름을 먼저 정리하려고 했습니다.`;
- `그래서 조회는 서버 컴포넌트, 변경은 Server Actions, 갱신은 revalidatePath와 router.refresh로 가져가서 서버를 기준으로 정합성을 맞추는 구조를 택했습니다.`;
- `상태도 무조건 전역으로 올리지 않고, 서버 상태와 로컬 UI 상태를 나눠서 복잡도를 줄이려고 했습니다.`;
- `지금 규모에서는 이 구조가 단순성과 정합성 측면에서 이점이 있었고, 규모가 더 커지면 캐시 계층이나 상태 관리 계층을 추가하는 방향으로 확장할 수 있다고 봤습니다.`;

1-2. 웹 표준, 시맨틱, 접근성

- 시맨틱 태그를 왜 쓰는지;
- 접근성을 교과서 수준이 아니라 실무에 어떻게 반영하는지;
- 키보드 포커스, 라벨, 알림, 모달 접근성, 확대 허용 같은 기본 기준이 있는지;

답변 예시:

- `이 프로젝트는 로그인 이후 쓰는 앱형 서비스라 초기 우선순위를 인증, 입력 경험, AI 파싱, 모바일 사용성에 더 뒀습니다.`;
- `그래서 접근성과 시맨틱 완성도는 에이전시 기준으로 부족한 편이고, 그 점은 먼저 인정하려고 합니다.`;
- `다만 몰라서 놓친 것보다는 우선순위를 뒤로 둔 것이고, 다시 한다면 시맨틱 랜드마크, 확대 허용, 포커스 이동, 모달 접근성, 라벨 연결부터 더 초기에 반영하겠습니다.`;
- `면접에서는 접근성을 옵션으로 본 게 아니라, 이번 프로젝트에서 뒤로 밀린 품질 항목으로 인식하고 있었다는 점을 분명히 말하는 게 중요합니다.`;

1-3. React/Vue 기반 컴포넌트 설계

- UI를 그냥 만드는 것이 아니라 재사용성과 확장성을 고려하는지;
- 상태와 표현을 어떻게 나누는지;
- 공통 컴포넌트 구조를 어떻게 생각하는지;

답변 예시:

- `UI를 한 번에 크게 묶기보다, 공통 레이아웃과 데이터 표시 영역, 입력 인터랙션 영역을 분리하려고 했습니다.`;
- `특히 shadcn/ui를 그대로 소비하는 데서 끝내지 않고, 서비스 맥락에 맞는 조합 단위로 감싸서 재사용 범위를 통제하려고 했습니다.`;
- `폼이나 드로어, 시트 같은 상호작용 요소는 표현과 상태를 최대한 분리해 교체 비용을 낮추는 쪽으로 생각했습니다.`;
- `제가 Vue와 Svelte 경험이 있었기 때문에, React에서도 결국 중요한 건 프레임워크 문법보다 상태 경계와 책임 분리라는 관점으로 접근했습니다.`;

1-4. 데이터 흐름과 API 협업

- 프론트가 단순 소비자가 아니라 데이터 구조와 API 계약을 이해하는지;
- 서버 상태와 클라이언트 상태를 구분할 수 있는지;
- 인증, 권한, 데이터 정합성을 구조 차원에서 설명할 수 있는지;

답변 예시:

- `프론트가 단순히 API를 소비하는 입장보다, 어떤 데이터 구조가 화면과 정합성에 유리한지 같이 고민하려고 했습니다.`;
- `가계부 도메인은 거래, 자산, 예산, 카테고리 관계가 명확해서 관계형 모델이 맞다고 봤고, 그래서 PostgreSQL 기반으로 갔습니다.`;
- `인증과 권한도 화면 분기 수준이 아니라 서버 경계에서 처리하는 게 맞다고 판단해서 Better Auth와 Server Actions 중심으로 묶었습니다.`;
- `거래 수정 하나가 여러 집계에 영향을 주는 구조라, API 편의성보다 데이터 일관성을 더 우선했습니다.`;

1-5. 성능과 크로스 브라우징

- 성능을 감이 아니라 구조와 근거로 설명하는지;
- 로딩, 캐싱, 스켈레톤, prefetch, 렌더링 범위를 이해하는지;
- 브라우저 차이와 호환성 이슈를 어떻게 대응하는지;

답변 예시:

- `성능은 단순히 빠르다기보다, 어디서 렌더링하고 어디서 데이터를 가져오느냐의 구조 문제라고 봤습니다.`;
- `그래서 가능한 조회는 서버에서 처리하고, 변경 이후에는 필요한 범위만 다시 갱신하는 쪽으로 설계했습니다.`;
- `사용자 체감 성능 측면에서는 로딩 상태, 스켈레톤, 페이지 전환 흐름, 모바일 입력 경험을 더 중요하게 봤습니다.`;
- `크로스 브라우징은 이번 프로젝트에서 가장 깊게 검증한 축은 아니지만, 실제 서비스로 가져간다면 입력 컴포넌트, 차트, 모바일 웹뷰 환경부터 우선 검증할 생각입니다.`;

1-5-1. 코드 근거로 설명할 수 있는지

- 추상적인 아키텍처 설명만이 아니라, 실제 코드에서 입력 검증, 폴백, 세션 처리, 파싱 방어 로직을 근거로 말할 수 있는지 볼 가능성이 있다.;
- 특히 공개 입력, 인증, `LLM` 기능이 있는 프로젝트라면 구현 세부를 모르면 금방 드러날 수 있다.;

답변 예시:

- `이 프로젝트는 공개 입력과 LLM 기능이 있어서, 추상적인 구조 설명만으로는 부족하다고 생각했습니다.`;
- `그래서 텍스트 입력 길이 제한, 이미지 MIME과 크기 검증, base64 위장 payload 차단, provider 폴백, 비정형 LLM 응답 파싱 방어 같은 부분을 직접 설명할 수 있게 준비하고 있습니다.`;
- `또 세션 IP와 UserAgent를 최소화해 저장한 이유, userId 범위로 데이터 접근을 묶은 이유도 보안과 개인정보 최소화 관점에서 설명할 수 있어야 한다고 봤습니다.`;
- `면접에서는 큰 아키텍처보다 이런 세부 판단을 제대로 이해하고 있는지가 더 빨리 드러난다고 생각합니다.`;

1-6. UX 관점의 문제 해결

- 시안 구현자보다 사용자 경험을 함께 고민하는 사람인지;
- 기획과 디자인을 그대로 받는 게 아니라 더 나은 구조를 제안할 수 있는지;

답변 예시:

- `이 프로젝트의 출발점 자체가 기록과 분류의 마찰을 줄이자는 문제의식이었습니다.`;
- `그래서 CRUD를 잘 만드는 것보다, 자연어 입력이나 이미지 입력으로 사용자의 입력 부담을 줄이는 데 더 집중했습니다.`;
- `또 모바일에서 자주 쓰는 서비스라고 보고, 앱처럼 빠르게 기록하고 수정할 수 있는 흐름을 우선했습니다.`;
- `즉 화면을 예쁘게 만드는 것보다, 어떤 마찰을 줄이고 어떤 행동을 더 쉽게 만들지에 초점을 둔 프로젝트라고 설명할 수 있습니다.`;

1-7. 운영 폭과 생산성

- 인증, 배포, 보안, AI 활용, 문서화 같은 운영 측면까지 이해하는지;
- AI 도구를 써도 책임과 검증을 사람이 가져가는지;

답변 예시:

- `이 프로젝트는 프론트 화면 구현에서 끝나지 않고, 인증, DB, 보안, AI 파싱, 모바일 확장성까지 같이 다뤘다는 점이 특징입니다.`;
- `구현 단계에서는 AI 도움을 많이 받았지만, 문제 정의, 구조 선택, 기술 채택, 검증 기준, 최종 승인 책임은 제가 가졌습니다.`;
- `저장소의 스킬 파일이나 파이프라인 문서는 AI에게 일을 떠넘긴 흔적이 아니라, 오히려 승인과 리뷰, QA 기준을 문서화해서 통제하려 한 흔적이라고 설명하려고 합니다.`;
- `결국 생산성 도구는 적극적으로 쓰되, 품질과 책임은 사람 쪽에 남겨두는 방식으로 작업했다는 점을 강조하면 됩니다.`;

---

2. 내 프로젝트와 연결되는 강점

- `React 19`, `Next.js 15`, `TypeScript` 기반이라 공고 스택과 직접 연결된다.;
- 로그인 기반 서비스라 서버 중심 데이터 흐름과 인증 구조를 설명하기 좋다.;
- `Supabase`, `Drizzle`, `Better Auth`, `Server Actions` 까지 연결해 화면만이 아니라 서비스 전반을 다뤘다는 점이 강점이다.;
- `LLM parse`, `parse API`, 보안 하드닝, rate limit, Origin 검증 같은 포인트는 일반 프론트 포트폴리오보다 넓은 범위를 보여준다.;
- `Capacitor` 와 모바일 퍼스트 구조는 실제 사용 환경을 고려했다는 점에서 강점이다.;
- `implementation-plan`, `history`, `code-quality`, `security-hardening` 같은 문서들은 생각 없이 구현하지 않았다는 근거가 된다.;

2-1. 기술 선택을 비교로 설명할 때의 포인트

Next.js 15

- 왜 선택했나: `RSC`, `Server Actions`, 서버/클라이언트 경계가 현재 프로젝트 구조와 잘 맞았기 때문;
- 비교 포인트: `SvelteKit`, `Remix` 도 가능했지만, 이번 프로젝트는 `React` 와 `Next.js` 를 실전으로 익히는 목적이 분명했다.;

Supabase + PostgreSQL

- 왜 선택했나: 관계형 모델, 집계 쿼리, 참조 무결성이 중요했기 때문;
- 비교 포인트: `Firebase` 의 장점은 실시간성과 빠른 프로토타이핑이지만, 이 도메인에서는 관계형 무결성과 집계가 더 중요했다.;

Drizzle

- 왜 선택했나: TypeScript 코드 자체가 스키마와 타입이 되어 개발 흐름이 단순했다.;
- 비교 포인트: `Prisma` 는 강력한 도구와 레퍼런스가 있지만, 당시에는 generate 단계와 구조 복잡도보다 단순성이 더 중요했다.;

Better Auth

- 왜 선택했나: 인증 흐름과 기존 DB 구조를 자연스럽게 연결하고, 가입 후 시딩이나 세션 저장 전 처리 같은 훅을 붙이기 쉬웠다.;
- 비교 포인트: `NextAuth` 는 표준성은 강하지만, 이번 프로젝트에서는 프로젝트 맞춤 로직 결합이 더 중요했다.;

---

3. 내 프로젝트에서 약하게 보일 수 있는 지점

- 웹표준, 시맨틱, 접근성, SEO 완성도는 에이전시 기준으로 약하다.;
- `Tailwind + shadcn/ui` 중심이라 공고의 `SCSS 기반 구조 설계` 와 직접 맞물리는 경험은 약하다.;
- `Zustand`, `React Query` 를 실제로 도입하지 않았기 때문에 상태관리 라이브러리 경험을 직접 증명하기는 어렵다.;
- `Vue`, `Svelte` 경험은 저장소만으로는 강하게 드러나지 않는다.;
- `CI/CD`, `AWS`, `BFF` 는 깊게 파고들면 방어해야 한다.;
- AI 사용 비중이 높고, 에이전트 스킬/파이프라인 문서가 많아 `직접 이해했는가` 질문이 나올 수 있다.;

4. 약점을 어떻게 설명할 것인가

4-1. 접근성/시맨틱/SEO

좋은 방향:

- `로그인 이후 사용하는 앱형 서비스였기 때문에 초기 우선순위를 인증, 거래 처리, AI 파싱, 정합성, 모바일 UX에 뒀다.`
- `몰라서 못 한 게 아니라 우선순위를 뒤로 둔 것이다.`
- `다시 한다면 시맨틱 구조, 확대 허용, 포커스 관리, 메타데이터를 더 초기에 반영하겠다.`

피해야 할 방향:

- `앱이라 접근성은 덜 중요했다.`
- `SEO는 필요 없었다.`
- `시간이 없어서 못 했다.`

4-2. Zustand / React Query 미도입

좋은 방향:

- `현재 구조에서 글로벌 상태가 크지 않았고, 서버 중심 흐름이 이미 정리돼 있었다.`
- `도입 자체보다 복잡도 대비 이득을 비교해서 의도적으로 넣지 않았다.`
- `규모가 커지면 도입 기준도 바뀔 수 있다.`

피해야 할 방향:

- `Next에서는 필요 없다.`
- `익숙하지 않아서 안 썼다.`

4-3. AI 사용 비중

좋은 방향:

- `AI가 구현 속도를 높였지만, 문제 정의, 구조 선택, 검증과 최종 책임은 제가 가졌다.`
- `손코딩 비율보다 설계 판단과 품질 통제가 더 중요하다고 봤다.`

피해야 할 방향:

- `거의 AI가 다 해서 저는 잘 모른다.`
- `동작하길래 그대로 썼다.`

5. 면접관이 실제로 확인하고 싶은 것

- 이 사람이 구조를 설명할 수 있는가;
- 선택한 기술의 장점뿐 아니라, 안 쓴 기술의 이유와 손실도 말할 수 있는가;
- 웹 표준과 접근성을 무시하는 사람이 아니라, 우선순위와 보완 계획을 이해하는 사람인가;
- AI를 써도 책임을 회피하지 않는가;
- 혼자 만든 프로젝트라도 협업 가능한 방식으로 사고하는가;

---

6. 우선순위 높은 예상 질문

1. 이 프로젝트를 왜 만들었나요?;
2. 왜 `React/Next` 를 배우는 프로젝트로 이 문제를 선택했나요?;
3. 왜 `App Router + Server Actions` 구조를 택했나요?;
4. 왜 `Supabase + PostgreSQL` 을 선택했나요?;
5. 왜 `Zustand` , `React Query` 를 넣지 않았나요?;
6. 왜 접근성/시맨틱/SEO가 약한가요?;
7. 왜 `Capacitor` 를 붙였나요?;
8. AI 파싱 결과는 얼마나 믿을 수 있나요?;
9. AI가 코드를 많이 썼다는데 본인 기여는 무엇인가요?;
10. 에이전트 스킬과 파이프라인 문서는 왜 만들었나요?;
11. 왜 `Firebase` 가 아니라 `PostgreSQL` 이었나요?;
12. 왜 `Prisma` 가 아니라 `Drizzle` 이었나요?;
13. 왜 `NextAuth` 가 아니라 `Better Auth` 였나요?;
14. `LLM` provider를 왜 여러 개 붙였나요?;
15. 텍스트 입력에 `base64` 위장 payload를 어떻게 막았나요?;
16. 이미지 입력은 왜 MIME, 크기, `base64` 를 따로 검증했나요?;
17. 세션 IP와 UserAgent는 왜 최소화했나요?;
18. `LLM` 응답이 JSON이 아니면 어떻게 처리했나요?;

---

7. 면접에서 가장 좋은 톤

- 과장하지 않는다.;
- 약점을 먼저 인정한다.;
- 변명 대신 우선순위를 설명한다.;
- 기술 선택은 항상 장점과 손실을 같이 말한다.;
- 마지막엔 `다시 하면 무엇을 바꾸겠다` 로 마무리한다.;

---

8. 짧은 정리

> 이 공고는 단순히 React를 쓸 줄 아는 사람보다, 구조 설계, 접근성, 데이터 흐름, 성능, UX 제안력까지 설명할 수 있는 프론트엔드를 보고 있다. 내 프로젝트는 앱형 서비스 중심이라 접근성/SEO는 약하지만, 서버 중심 구조, 관계형 데이터 모델링, 인증과 보안, AI 기능 연결, 문서화와 검증 기준 면에서 충분히 강점을 만들 수 있다.;